

中間レポート補助テキスト

[[5, 1, 3]] 自作デコーダ on-ramp

量子コンピューティング I

2026 年 6 月 27 日

本書は、中間レポートの土台でつまづいたときに開くための補助テキストです。課題そのもの ([[5, 1, 3]] の自作デコーダ) ではなく、もっと小さな別の符号で「やり方」だけを最後まで通します。やり方を [[5, 1, 3]] に当てはめる部分は、採点の対象であり、本書では扱いません。

第 1 章

オリエンテーション：何が配られ、何を作るのか

この補助テキストは、中間レポートの土台でつまずいたときに開くためのものです。レポートの課題は「 $[[5, 1, 3]]$ 符号の自作デコーダ」を組み立てるのですが、いきなり全体に手をつけようとすると、どこから始めればよいかが見えにくくなります。そこで本書では、課題そのもの ($[[5, 1, 3]]$) ではなく、もっと小さな別の符号を使って「やり方」だけを最後まで通します。やり方さえ手に入れば、 $[[5, 1, 3]]$ への当てはめは自分の手で進められます。そして、その当てはめの部分こそが採点の対象です。だから本書は、当てはめには踏み込みません。

まず、課題で「配られるもの」と「自分で作るもの」を切り分けておきましょう。

■配られるもの (手本・道具として使う).

- エンコーダ —— 論理 1 ビット分の情報を 5 つの物理量子ビットに符号化する回路。導出は講義の範囲を超えるので、完成形が渡されます。テスト用の状態を作るのに使うだけです。
- 生成子 g_1 – g_4 —— 符号を定義する 4 本のスタビライザ。
- 論理演算子 —— 符号の中で論理ビットを操作する演算子。
- syndrome の見本 (1 ケース) —— ある誤りに対して syndrome をどう求めるか、その例が 1 つだけ示されます。

■自分で作るもの (採点対象).

- デコーダ —— 「syndrome を見て、どの誤りを当てて戻すか」の対応表 (15

通り)。これが課題の中心です。

- 単一誤り訂正の実証 —— 作った表が、実際に単一の誤りを直せることを確かめます。
- [発展 A] 符号化が有利になる物理誤り率の境目 (break-even) を求める。
- [発展 S] CX 誤り・耐故障性に関わる部分。

本書が足場を組むのは、単位ラインにあたるデコーダと単一誤り訂正、それに発展 A の「やり方の入口」までです。発展 S には手を出しません。そこは丸ごと課題に残します。

次に、そのデコーダが何を読み取っている装置なのか、最初の足場を置きます。鍵になるのは「符号空間」という考え方です。

生成子 (スタビライザ) は、符号として正しい状態をそのまま固定する演算子です。正しい状態に生成子をかけても、状態は変わりません (固有値は $+1$)。すべての生成子が同時に $+1$ で固定する状態の集まり —— これが符号空間です。論理状態 $|0_L\rangle$ と $|1_L\rangle$ は、この符号空間の基底にあたります。言い換えれば、論理状態は符号空間の中に住んでいます。

ここで誤りが起きると、状態は符号空間の外へ押し出されます。すると、さっきまで $+1$ で固定していた生成子のうち、いくつかは $+1$ でなくなります。この「どの生成子が $+1$ から外れたか」というパターンを読み取ることが、誤り訂正の出発点です。読み取り方そのものは第3章で具体的に組み立てます。本章では、「符号空間から外れた生成子を読む」という見取り図だけ持っておけば十分です。

最後に、本書の進め方をまとめます。やり方は、3 量子ビット符号という最小の符号で最後まで通し、必要なところで Steane 符号を引き合いに出します。[[5, 1, 3]] そのものは構造の確認にとどめ、syndrome の計算はしません。手に入れたやり方を [[5, 1, 3]] に当てはめるのは、レポートでの自分の作業です。それでは次章から、その「やり方」を一段ずつ積み上げていきましょう。

第 2 章

スタビライザが符号を定義する 仕組み (3 量子ビット符号で)

第 1 章では「符号空間」を、生成子が $+1$ で固定する状態の集まり、という形で抽象的に置きました。本章では、それを最小の符号 —— 3 量子ビット符号 —— で具体的にします。見るのは 2 つです。エンコーダが論理ビットをどうやって符号空間に乗せるか、そして生成子がその符号空間をどう定めるか。

まず回路から見ます (先にコード、あとで数式)。3 量子ビット符号は、1 つの論理ビットを 3 つの物理量子ビットに広げます。符号化したい状態を q_0 に置き、 q_1 と q_2 は $|0\rangle$ にしておきます。ここに、 q_0 を制御・ q_1 を標的とする CNOT と、 q_0 を制御・ q_2 を標的とする CNOT を、順にかけます。たった 2 つのゲートです。

この回路の働きを、基底ごとに追ってみましょう。CNOT は制御が $|1\rangle$ のときだけ標的を反転させます。

- 入力が $|0\rangle|0\rangle|0\rangle$ のとき： q_0 が $|0\rangle$ なので、どちらの CNOT も標的を反転させません $\rightarrow |000\rangle$ のまま。
- 入力が $|1\rangle|0\rangle|0\rangle$ のとき： q_0 が $|1\rangle$ なので、両方の CNOT が標的を反転させます $\rightarrow |111\rangle$ 。

したがって、符号化前の状態を $\alpha|0\rangle + \beta|1\rangle$ とすると、符号化後は $\alpha|000\rangle + \beta|111\rangle$ になります。論理ゼロは $|0_L\rangle = |000\rangle$ 、論理イチは $|1_L\rangle = |111\rangle$ 。これがこの符号の論理状態です。

エンコーダの「導き方」の勘どころは、 $|0\rangle$ を $|0_L\rangle$ に、 $|1\rangle$ を $|1_L\rangle$ に移しつつ、情報を複数の量子ビットに広げる回路を組む、という点にあります。3 量子ビット

符号では、 q_0 の値を $q_1 \cdot q_2$ に行き渡らせる 2 つの CNOT がちょうどその役割を果たします。

つぎに、この $\alpha|000\rangle + \beta|111\rangle$ たちが「符号空間」だと、第1章の生成子の言葉で確かめます。3 量子ビット符号の生成子は $S_1 = ZZI$ と $S_2 = IZZ$ の 2 本です。Z は、 $|0\rangle$ はそのまま、 $|1\rangle$ には負符号をつけて $-|1\rangle$ にする演算子です。これを使うと：

- $|000\rangle$ に $S_1 = ZZI$ をかけると、 $(+1)(+1)$ で全体は $+1$ 。
- $|111\rangle$ に $S_1 = ZZI$ をかけると、 $(-1)(-1)$ で全体はやはり $+1$ 。

$S_2 = IZZ$ でも同じく、 $|000\rangle$ も $|111\rangle$ も $+1$ になります。つまり $\text{span}\{|000\rangle, |111\rangle\}$ は、 S_1 と S_2 が同時に $+1$ で固定する空間そのもの——第1章でいう符号空間です。生成子がびたりとこの 2 状態を選び出している、と読めます。

ここで $S_1 = ZZI$ が何を見ているかを直感で言うと、 q_0 と q_1 の値が「そろっているか」です (そろっていれば $+1$)。 $S_2 = IZZ$ は q_1 と q_2 がそろっているかを見ます。符号の正しい状態 $|000\rangle \cdot |111\rangle$ はすべての桁がそろっているので、どちらの生成子も $+1$ になる、というわけです。では、もし 1 つの桁が反転したら生成子はどう答えるのか——それが復号の核心で、次章で組み立てます。

最後に $[[5, 1, 3]]$ について一言。 $[[5, 1, 3]]$ のエンコーダはこれよりずっと込み入っていて、その導き方は本講義の範囲を超えます。だから完成形がスターターノートブックで配られます。これを自分で作る必要はありません。テスト用の $|0_L\rangle \cdot |1_L\rangle$ を用意するために使うだけで、採点の対象でもありません。

これで、符号空間と、そこへ論理状態を乗せるエンコーダがそろいました。次章では、誤りが状態を符号空間の外へ押し出したとき、それがどの誤りだったかをどう読み取るか——デコーダの核心に入ります。

第 3 章

復号の原理：syndrome は「反交換する生成子の集まり」

第 2 章では、生成子が符号空間を $+1$ で固定することを確認しました。本章は本書の心臓です。誤りが状態を符号空間の外へ押し出したとき、それが「どの誤りだったか」を生成子に問い合わせて読み取る —— デコーダの原理を、3 量子ビット符号で最後まで通します。読み取りの鍵は、たった 1 つの問いに集約されます。「2 つの演算子は、掛ける順序を入れ替えてよいか」。

3.1 可換性の素

2 つの演算子 A, B について、 AB と BA が等しいとき、2 つは可換 (交換できる) といいます。等しくならず、符号だけが反転して $AB = -BA$ となるとき、反可換といいます。Pauli 演算子 I, X, Y, Z のどの 2 つも、必ずこのどちらかになります —— 中間はありません。

まず 1 つの量子ビットの上で、どの組が可換でどの組が反可換かを見ます。規則は単純です。

- I はすべてと可換です (I は何もしない演算子なので、順序を入れ替えても影響しません)。
- 同じ Pauli 同士は可換です (X と X 、 Y と Y 、 Z と Z)。
- 異なる非自明な Pauli 同士 (X と Y 、 Y と Z 、 Z と X) は反可換です。たとえば $XZ = -ZX$ 。

表にすると次の通りです (表 3.1)。この表は手元に置いておいてください。本章でも次章でも、何度も引きます。

表 3.1 1 量子ビット上の Pauli 可換性 (○ = 可換, × = 反可換)

	I	X	Y	Z
I	○	○	○	○
X	○	○	×	×
Y	○	×	○	×
Z	○	×	×	○

3.2 複数の量子ビットへ

量子ビットが複数になっても、考え方は変わりません。2つの多量子ビット Pauli 演算子を見比べるときは、各位置 (各量子ビット) ごとに上の表で可換か反可換かを判定し、反可換だった位置の数を数えます。

- 反可換の位置が奇数個なら、全体として反可換。
- 反可換の位置が偶数個 (0 個を含む) なら、全体として可換。

理由はこうです。反可換の位置が 1 つあるたびに、順序を入れ替えると符号 -1 が 1 つ出ます。位置をまたいで掛け合わせると、符号は -1 を反可換の個数だけ掛けたものになります。式で書けば、 $A = \bigotimes_i A_i$ 、 $B = \bigotimes_i B_i$ に対して

$$AB = (-1)^k BA, \quad k = \#\{i : A_i \text{ と } B_i \text{ が反可換}\}$$

であり、 -1 を奇数回掛ければ -1 、偶数回なら $+1$ 。だから「反可換の位置の数の偶奇」だけで全体が決まります (A, B が可換 $\iff k$ が偶数)。符号を数える、と覚えてください。

3.3 syndrome の定義

ここで誤りと生成子の関係を式で押さえます。符号化された状態 $|\psi\rangle$ は、各生成子 g に対して $g|\psi\rangle = |\psi\rangle$ 、つまり $+1$ で固定されていました。いま誤り E が起きて、状態が $E|\psi\rangle$ になったとします。この状態に生成子 g を当てると $gE|\psi\rangle$ ですが、ここで g と E の順序を入れ替えます。

- g と E が可換なら、 $gE = Eg$ なので、 $gE|\psi\rangle = Eg|\psi\rangle = E|\psi\rangle$ 。生成子はこの状態を $+1$ で固定したままです。
- g と E が反可換なら、 $gE = -Eg$ なので、 $gE|\psi\rangle = -Eg|\psi\rangle = -E|\psi\rangle$ 。生成子の固定が -1 にひっくり返ります。

つまり各生成子 g_i は、誤り E に対して

$$g_i E|\psi\rangle = (-1)^{s_i} E|\psi\rangle, \quad s_i \in \{0, 1\}$$

を満たし、 $+1$ (変わらず, $s_i = 0$) か -1 (反転, $s_i = 1$) のどちらかを返します。この $0/1$ を生成子ごとに並べたビット列が syndrome です。鍵は、syndrome を決めているのが「生成子と誤りが可換か反可換か」だけ、という点です。

3.4 3 量子ビット符号で計算する

では3量子ビット符号で、実際に syndrome を求めます。生成子は $S_1 = ZZI$ と $S_2 = IZZ$ 。直したい誤りは、3か所のどれか1つでのビット反転です。位置1なら $X_1 = X \otimes I \otimes I$ 、位置2なら $X_2 = I \otimes X \otimes I$ 、位置3なら $X_3 = I \otimes I \otimes X$ と書きます。

まず $X_1 = XII$ の syndrome を、可換性の表を1つずつ当てて求めます。 $S_1 = ZZI$ と $X_1 = XII$ を位置ごとに見比べます。

- 位置1: Z と $X \rightarrow$ 反可換 ($\times$)。
- 位置2: Z と $I \rightarrow$ 可換 ($\bigcirc$)。
- 位置3: I と $I \rightarrow$ 可換 ($\bigcirc$)。

反可換は1か所、奇数なので S_1 全体は反可換 $\rightarrow S_1$ の応答は1。次に $S_2 = IZZ$ と $X_1 = XII$ 。

- 位置1: I と $X \rightarrow$ 可換。
- 位置2: Z と $I \rightarrow$ 可換。
- 位置3: Z と $I \rightarrow$ 可換。

反可換は0か所、偶数なので可換 $\rightarrow S_2$ の応答は0。これで X_1 の syndrome は $(S_1, S_2) = (1, 0)$ と決まりました。

同じ手順を X_2, X_3 にも当てます。 $X_2 = IXI$ は、 $S_1 = ZZI$ とは位置2で Z と X が反可換(1か所) $\rightarrow S_1$ 応答1、 $S_2 = IZZ$ とは位置2で Z と X が反可換(1か所) $\rightarrow S_2$ 応答1。よって $(1, 1)$ 。 $X_3 = IIX$ は、 S_1 とは反可換の位置が0(位置3は

I と X で可換)→ S_1 応答 0、 S_2 とは位置 3 で Z と X が反可換 (1 か所)→ S_2 応答 1。よって (0, 1)。誤りがまったく起きていなければ両生成子とも +1 のまま、syndrome は (0, 0) です。まとめると表 3.2。

表 3.2 3 量子ビット符号：単一ビット反転の syndrome

誤り	syndrome (S_1, S_2)
なし	(0, 0)
X_1	(1, 0)
X_2	(1, 1)
X_3	(0, 1)

4 つの syndrome はすべて異なります。だから syndrome を見れば、どの誤りが起きたか (あるいは起きていないか) が 1 通りに定まる —— これが表引きの土台です。

3.5 列読み —— 計算を速くする見方

いま求めた syndrome を、生成子を行・誤り位置を列にした表に並べ替えてみます (表 3.3)。

表 3.3 列読み：生成子 (行)× 誤り位置 (列)

	X_1	X_2	X_3
$S_1 = ZZI$	1	1	0
$S_2 = IZZ$	0	1	1

S_1 の行 (1, 1, 0) を見てください。これは $S_1 = ZZI$ の中身をそのまま反映しています。X 誤りは Z と反可換・I と可換なので、Z がある位置 (1, 2) では 1、I の位置 (3) では 0。 $S_2 = IZZ$ の行 (0, 1, 1) も同じで、Z のある位置 (2, 3) が 1 です。

そして各誤りの syndrome は、この表のその誤りの列を縦に読むだけで得られます。 X_1 なら 1 列目 (1, 0)、 X_2 なら 2 列目 (1, 1)、 X_3 なら 3 列目 (0, 1)。生成子を「X 誤りが反応する位置に 1 が立つビット列」として書いておけば、単一の誤りの syndrome はもう計算せずにその列を読むだけ —— これを列読みと呼びます。次章で生成子が増えても、この見方がそのまま効きます。

3.6 訂正まで一周つなぐ

ここまでの部品 (可換性 $\rightarrow$ syndrome $\rightarrow$ 表) を、訂正までつないで一度通します。

1. 表を作る (表 3.2 の 4 行)。これがデコーダです。
2. 誤りが起きる。ここでは位置 2 のビット反転 X_2 が起きたとします。
3. 生成子 S_1, S_2 を測定して syndrome を読む。結果は $(1, 1)$ 。
4. 表を引く。 $(1, 1)$ に対応する誤りは X_2 。
5. 訂正する。 X_2 をもう一度かけます。 X は $XX = I$ を満たす (同じ反転を 2 回かけると元に戻る) ので、誤りがちょうど打ち消されます。
6. 確認する。状態は符号空間に戻り (両生成子が再び $+1$)、論理状態も元のまま保たれています。

これで「syndrome を読む $\rightarrow$ 表を引く $\rightarrow$ 訂正する」という 1 周が、具体的な符号で最後まで通りました。デコーダがやっているのは、結局この表引きです。

3.7 この先へ

最後に 2 つ、ことわっておきます。1 つめ。3 量子ビット符号が直せるのは X (ビット反転) の誤りだけです。 Z や Y の誤りは、この符号では区別も訂正もできません。これはこの小さな符号の限界です。距離 3 の $[[5, 1, 3]]$ は X も Z も Y も訂正できる符号で、その「3 種類の誤りをどう区別するか」が次章の主題です。本章で $X \cdot Y \cdot Z$ を一度に詰め込まないのは、まず X だけで読み方の原理を固めるためです。

2 つめ。いま組み立てた原理 —— 可換性で各生成子の応答を求め、表にまとめ、表引きで訂正する —— は、 $[[5, 1, 3]]$ でもそっくりそのまま使えます。違うのは、生成子が 4 本に増え、扱う誤りの種類も増えることだけです。ただし $[[5, 1, 3]]$ の syndrome をここで計算することはしません。それは、配られる 1 ケースの見本を手本に、可換性の表を当てて自分で埋めていく —— まさにレポートの中心の作業だからです。次章では、その作業に入る前の最後の橋 —— CSS と非 CSS の違い —— を渡します。

第 4 章

X だけでなく $Y \cdot Z$ も : CSS と非 CSS の違い

第 3 章では、3 量子ビット符号で「syndrome を読んで表を引く」原理を最後まで固めました。ただし、あの符号が直せるのは X の誤りだけでした。レポートの $[[5, 1, 3]]$ は距離 3 の符号で、 X も Z も Y も直します。本章の主題は 1 つ、「3 種類の誤りをどう区別するか」です。そしてこの章で、 $[[5, 1, 3]]$ という符号に橋を渡します。もっとも間違えやすいのが、ここです。

4.1 生成子の応答を、 X だけから $X \cdot Y \cdot Z$ へ

第 3 章の列読みは、 X 誤りだけを相手にしていました。生成子の Z がある位置に X 誤りが来れば反応する (反可換 $\rightarrow 1$)、という話です。これを $Y \cdot Z$ にも広げます。広げるといっても、新しい規則も新しい表も要りません。第 3 章の可換性の表 (表 3.1) を、誤り側を $X \cdot Y \cdot Z$ に絞って読み直すだけです ($\bigcirc$ が syndrome bit 0、 $\times$ が 1 でした)。

その表を、生成子のその位置の Pauli ごとに 1 行ずつたどってみます。 X を持つ生成子の行を見ると、 X 誤りとは可換 (bit 0)、 Y 誤り・ Z 誤りとは反可換 (bit 1) です。 Z を持つ生成子はちょうど逆で、 Z 誤りに 0、 $X \cdot Y$ 誤りに 1 を返します。 Y を持つ生成子は、 Y 誤りに 0、 $X \cdot Z$ 誤りに 1 です。

ここで注目してほしいのは、同じ位置でも、 X 誤りと Z 誤りが別のビットを立てる点です。いま読んだ通り、 X を持つ生成子は X 誤りを素通しし (0)、 Z 誤りを捕まえます (1)。 Z を持つ生成子はその逆でした。 Y は、 X と Z とともに反応する「両

刀」の誤りで、Y は X と Z が同じ場所に同時に起きたようなもの (Y は X と Z の積に比例し、両方の性質を併せ持つ)、とっておくと見通しがよくなります。この「X 用と Z 用の役割分担」が、次の話の鍵になります。

4.2 CSS 符号：X と Z を切り離して解ける符号 (Steane、既習)

符号の中には、生成子を「純粋に X だけからなる群」と「純粋に Z だけからなる群」の2つにきれいに分けられるものがあります。これを CSS 符号と呼びます。第11回で扱った Steane 符号 ($[[7, 1, 3]]$) が、その代表例です。

この分け方ができると、復号がとても楽になります。さきほどの読み取りを思い出してください。純 X 群の生成子は、X 誤りには反応せず (X と X は可換 $\rightarrow 0$)、Z 誤りにだけ反応します。純 Z 群はその逆です。つまり、

- X 誤りは、純 Z 群の生成子だけが拾う。
- Z 誤りは、純 X 群の生成子だけが拾う。

X 誤りと Z 誤りが、まったく別の生成子に振り分けられるのです。だから「X 誤り用の表引き」と「Z 誤り用の表引き」を**独立**に走らせれば済みます。1つの大きな問題が、2つの小さな問題に分かれる——これが CSS 符号の強みです。Steane についてはこの構図を確認するだけで十分なので、ここで解き直すことはしません。

4.3 非 CSS の $[[5, 1, 3]]$ ：その分け方ができない

ところが $[[5, 1, 3]]$ は、この分け方ができません。生成子の1本1本の中に、X と Z が**混ざっている**からです。たとえば最初の生成子は $g_1 = XZZXI$ で、X も Z も含んでいます (残りの g_2 – g_4 も同じように混合で、4本の具体形はスターターノートブックで配られます)。

1本の生成子に X と Z が同居していると、「この生成子は X 誤り専用」「あの生成子は Z 誤り専用」という振り分けができません。どの生成子も、位置によって X 誤りにも Z 誤りにも反応します。結果として、CSS の分離復号——X と Z を別々に解く方法——は使えません。

ここが、もっとも間違えやすいところです。

注意 (最大の落とし穴). Steane で身についた「X と Z を分けて解く」発想を、そのまま $[[5, 1, 3]]$ に持ち込まないでください。 $[[5, 1, 3]]$ では、1 つ 1 つの単一誤りについて、4 本の生成子すべてに対する応答 (4 ビット) をまとめて求め、1 枚の表に畳みます。分離はしません。

とはいえ、やること自体は第3章で固めた原理のままです。可換性で各生成子の応答を求め、表に並べ、表引きで訂正する。変わったのは、生成子が4本に増え、誤りの種類が $X \cdot Y \cdot Z$ の3つになっただけです。

4.4 なぜ 15 通りで「ちょうど」埋まるのか：完全符号

ここで、デコーダの表がなぜ 15 行になるのかを、数の上で確かめておきます。

直すべき単一誤りは、5 つの位置のそれぞれに $X \cdot Y \cdot Z$ の 3 種類なので、 $5 \times 3 = 15$ 個あります。一方、生成子は 4 本で、各本が 0 か 1 を返すので、syndrome のパターンは $2^4 = 16$ 通り。このうち「誤りなし」が $(0, 0, 0, 0)$ を占めるので、残りの**非ゼロ**の syndrome は $16 - 1 = 15$ 通りです。

15 個の単一誤りと、15 通りの非ゼロ syndrome。数がぴたりと一致します。だから単一誤りは 1 個ずつ別々の syndrome に収まり、表が過不足なく埋まる——このような符号を完全符号と呼びます。 $[[5, 1, 3]]$ が完全符号であることが、15 行ちょうどのデコーダが作れる理由です。

ただし、ここで言えるのは「数が合う」ところまでです。どの誤りがどの syndrome に対応するか——つまり表の中身そのもの——は、可換性を 1 個ずつ当てて自分で埋めます。それがデコーダ作りの本体です。

4.5 ここから課題へ

道具はそろいました。 $[[5, 1, 3]]$ のデコーダを作るとは、結局こういう作業です。配られた生成子 g_1 – g_4 と、見本として配られる syndrome 1 ケースを手本にして、残り 14 個の単一誤りそれぞれに、第3章の列読み (を $X \cdot Y \cdot Z$ に広げたもの) を当てて 4 ビットの syndrome を求め、「どの syndrome なら、どの誤りを当てて戻すか」の対応表を 1 枚に畳む——これがデコーダです。

本書で渡したものを並べておきます。符号空間という土台 (第1章)、エンコーダと固定の仕組み (第2章)、可換性・syndrome・表引き・列読み (第3章)、そして CSS と非 CSS の違いと、混ざった生成子の扱い (本章)。あとは、これを $[[5, 1, 3]]$ に当てはめるだけです。

次章では、できあがったデコーダが**どれだけ誤りに強い**かを確かめる方法に進みます。表が正しく動くことを越えて、符号化が割に合うのはどんなときか——発展Aの入口です。

第 5 章

耐性の確かめ方： p を振って論理誤り率を見る

第 3 章・第 4 章で、デコーダの「作り方」がそろいました。表が単一の誤りを正しく直せること——これは確かに必要です。けれど、それだけでは「この符号は誤りに強いのか」までは分かりません。本章では、できあがったデコーダがどれくらい誤りに耐えるかを測る方法を、3 量子ビット符号で通します。これが発展 A の入口です。

5.1 論理誤り率 P_L とモンテカルロ推定

測りたいのは、論理誤り率 P_L です。物理量子ビットの 1 つ 1 つは、確率 p で誤ります。この p に対して、訂正してもなお論理ビットが反転してしまう確率が P_L です。 p が小さいほど P_L も小さくなりますが、その下がり方こそが符号の強さを表します。

P_L を厳密な式で出すのが難しい符号も多いので、一般には数え上げではなくモンテカルロ推定を使います。考え方は単純で、「誤りをたくさんサンプルして、失敗の割合を数える」だけです。1 回の試行はこう進みます。各物理量子ビットに確率 p で誤りを置く → その誤りの syndrome を可換性で求める → 自作の表で復号する → 論理状態が反転したかを判定する。これを N 回繰り返し、失敗した回数を N で割れば、 P_L の推定値が得られます。

言語によらない擬似コードで書くと、次の形です。

```
fail ← 0
```

```
N 回くりかえす:
```

```
    error ← 各物理量子ビットに確率  $p$  で誤りを置く (3 量子ビット符号ではビット反転)
```

```
    s ← syndrome(error) (生成子ごとの 0/1 を可換性で求める)
```

```
    guess ← table[s] (自作デコーダの表引き)
```

```
    error に guess を当てて訂正する
```

```
    もし論理状態が反転していたら: fail ← fail + 1
```

```
 $P_L \leftarrow \text{fail} / N$ 
```

実際の量子回路上での実装 (誤りの注入や測定のためのハーネス) はスターターノートブックの領域なので、ここでは手続きの骨格だけを示しました。

5.2 交差点 (break-even) を探す

符号を使うのが得かどうかは、符号化したときの P_L を、符号化しないときの誤り率 p と並べて比べると見えます。横軸を p 、縦軸を誤り率にとり、2 本の線を重ねます。

- 符号化しない線：そのまま p (45 度の直線)。
- 符号化する線： P_L 。

p が小さいうちは符号化した線のほうが下にいて ($P_L < p$)、符号化が得をしています。ところがある点で2本の線が交わり、それより p が大きくなると符号化した線のほうが上に出てしまう ($P_L > p$)。この交わる点を **break-even(損益分岐点)** と呼びます。break-even より誤りの少ない領域でだけ、符号化は割に合います。

5.3 $\sim p^2$ の見立て (健全性チェック)

数値を出す前に、答えの「形」を見積もっておくと、計算が正しい方向に進んでいるかを確認できます。3 量子ビット符号は距離 3 で、単一の誤りは訂正できます。だから論理的な失敗が起きるには、誤りが **2 個以上** 必要です。確率の最低次は、誤り 2 個分の p^2 です。したがって、小さい p では

$$P_L \approx cp^2$$

の形になるはずですが。実際にプロットや推定値の立ち上がりが p^2 主導になっていれば、健全。 p の 1 次 (cp) で立ち上がっていたら、どこかで単一誤りの訂正に失

敗しています。

5.4 3 量子ビット符号で具体的に

3 量子ビット符号は、論理的な失敗 (多数決が外れること) が、3 つの物理量子ビットのうち 2 個以上が反転したときに起きます。ビット反転チャンネルでこの確率を数えると、係数を明示して書けば

$$P_L = \binom{3}{2}p^2(1-p) + \binom{3}{3}p^3 = 3p^2(1-p) + p^3 = 3p^2 - 2p^3$$

になります (係数の 3 は「3 か所のうち 2 個が反転する組み合わせの数」 $\binom{3}{2} = 3$ 、 p^3 の項は 3 個全部が反転する場合の補正です)。小さい p では先頭の $3p^2$ が効き、さきほどの $\sim p^2$ の見立て ($c = 3$) と合っています。

break-even は、この P_L が符号化しない p と等しくなる点なので、 $P_L = p$ を解きます。

$$\begin{aligned} 3p^2 - 2p^3 &= p \\ 3p - 2p^2 &= 1 && (\text{両辺を } p \text{ で割る}) \\ 2p^2 - 3p + 1 &= 0 \\ (2p - 1)(p - 1) &= 0 \end{aligned}$$

意味のある解は $p = \mathbf{0.5000}$ です ($p = 1$ は全反転で、符号の話として意味がありません)。つまり 3 量子ビット符号は、物理誤り率が 0.5000 を下回る領域で符号化が得をする、と読めます。透明な小例で、break-even が実際にどう求まるかが見えました。

5.5 $[[5,1,3]]$ へ

注意. この **0.5000** は **3 量子ビット符号の値**であって、 $[[5,1,3]]$ の値ではありません。 $[[5,1,3]]$ の break-even は、あなたが自分で求めるものです。

手法はそのまま使えますが、1 点だけ変わります。3 量子ビット符号の誤りはビット反転 (X) だけでしたが、 $[[5,1,3]]$ が想定するのは depolarizing チャンネルで、各量子ビットに $X \cdot Y \cdot Z$ のいずれかが起こりえます。だから上の擬似コードの「誤りを置く」ステップで、X だけでなく $X \cdot Y \cdot Z$ をサンプルします。あとの流れ (syndrome $\rightarrow$ 表引き $\rightarrow$ 論理反転の判定 $\rightarrow$ 失敗の集計) は、3 量子ビット符号の

ときと同じです。[[5, 1, 3]] には $3p^2 - 2p^3$ のようなきれいな式は望みにくいので、モンテカルロで2本の線を重ね、交差点を数値で読み取る——それが [[5, 1, 3]] の break-even を出す道筋です。

第 6 章

ここから先は自分で

本書はここまで、 $[[5, 1, 3]]$ そのものではなく、もっと小さな符号で「やり方」を渡してきました。最後に、その手放しの地点を確認します。いま、あなたの手に残されている作業は、次の通りです。

- ① **デコーダを作る**. 配られた生成子 g_1 – g_4 に、第 3 章・第 4 章の可換性規則を当てて、15 個の単一誤りそれぞれの syndrome を求め、「syndrome → 当てるべき誤り」の対応表を 1 枚に畳む。
- ② **訂正を実証する**. 作った表が、15 通りの単一誤りを実際に直せることを確かめる。
- ③ **〔発展 A〕強さを測る**. 第 5 章の手法 (p を振り、 P_L を推定し、非符号化と重ねて交差点を探す) を $[[5, 1, 3]]$ に当て、その break-even を求める。
- ④ **〔発展 S〕CX 誤り・耐故障性**. ここは本書では一切扱っていません。完全に開いたまま、あなたの手に残されています。

本書が渡したのは、別の符号で練習した「手つき」だけです。 $[[5, 1, 3]]$ に固有のもの——その 15 行の表、その break-even の値、その先に待つもの——は、本書には載っていません。それらは課題が問うている発見であり、あなた自身が見つけるものです。第 3 章で一度通した「syndrome を読む → 表を引く → 訂正する」を、今度は $[[5, 1, 3]]$ の生成子で、自分の手で一周してみてください。土台はもう、そろっています。